

Отчёт по домашнему заданию №3

Пет-проект: классическое ML + нейронные сети

Детекция токсичных комментариев

Датасет: Jigsaw Toxic Comment Classification Challenge (Kaggle)

1. Цель и постановка задачи

Цель проекта — построить и сравнить модели мульти-лейбл классификации текстовых комментариев по 6 независимым (не взаимоисключающим) меткам токсичности: `toxic`, `severe_toxic`, `obscene`, `threat`, `insult`, `identity_hate`. Каждый комментарий может одновременно относиться к нескольким классам, поэтому задача решается как 6 независимых бинарных классификаций (One-vs-Rest), а не как задача с одним классом на объект.

В работе исследуется как классический ML-подход (TF-IDF + Logistic Regression / Random Forest / Gradient Boosting), так и нейросетевой подход (MLP на Keras), с последующим сравнением по единым метрикам и подбором индивидуального порога классификации для каждого лейбла.

2. EDA и предобработка

2.1 Описание датасета

Обучающая выборка Jigsaw Toxic Comment Classification содержит около 159 000 комментариев с англоязычных страниц обсуждений Wikipedia. Единственный содержательный признак — сырой текст комментария (`comment_text`); целевая переменная — 6 бинарных меток токсичности. Пропусков в тексте нет, но встречаются пустые/технические строки, которые были отфильтрованы на этапе предобработки.

2.2 Баланс классов

Классы сильно несбалансированы: подавляющее большинство комментариев (около 90%) не токсичны ни по одному лейблу. Среди токсичных доминирует класс `toxic` (~9-10% от всех комментариев), а редкие классы `threat` и `severe_toxic` встречаются менее чем в 1-1.5% случаев. Такой дисбаланс — главная причина, по которой `accuracy/subset_accuracy` как метрика вводит в заблуждение, поэтому в работе в качестве основных метрик используются `macro ROC-AUC` и `macro F1`, устойчивые к дисбалансу классов.

2.3 Корреляция между лейблами

Матрица корреляций показывает выраженную положительную связь между `toxic`, `obscene` и `insult` — грубые/оскорбительные комментарии почти всегда одновременно помечены как `toxic`. Классы `threat` и `identity_hate` слабее коррелируют с остальными и оказались наиболее сложными для моделей (см. раздел 5), что согласуется с их малой частотой и более разнородным лексическим наполнением.

2.4 Обработка пропусков, выбросов и очистка текста

- Удалены полностью пустые/технические строки комментариев.
- Длина комментария обрезана по 99.5 перцентилю — экстремально длинные тексты не позволяют TF-IDF адекватно взвешивать редкие термины.
- Текст приведён к нижнему регистру, удалены URL, служебные символы, лишние пробелы (при этом сохранены “!”, “?”, апострофы — они несут сигнал эмоциональности).

2.5 Feature engineering

Помимо TF-IDF-векторизации текста, созданы 4 дополнительных мета-признака с обоснованием:

- `caps_ratio` — доля заглавных букв в тексте; капс часто используется для передачи агрессивного тона.

- `exclaim_count` — количество восклицательных знаков; эмоционально заряженные (в т.ч. токсичные) сообщения чаще используют “!!!”.
- `unique_word_ratio` — отношение уникальных слов к общей длине; повторяющаяся брань/спам снижает лексическое разнообразие.
- `bad_word_count` — количество слов из списка бранной лексики; прямой лексический сигнал токсичности.

Как показал анализ важности признаков, `bad_word_count` и `caps_ratio` оказались одними из самых значимых признаков для модели Random Forest - то есть инженерные мета-признаки внесли реальный вклад, а не только TF-IDF-токены.

3. Классические ML-модели

OneVsRestClassifier (независимая модель на каждый из 6 лейблов): линейная модель Logistic Regression, дерево-based модель Random Forest и ансамблевый градиентный бустинг LightGBM.

3.1 Подбор гиперпараметров

Гиперпараметры каждой модели подбирались с помощью RandomizedSearchCV (scoring = ROC-AUC, 5-фолдовая внутренняя кросс-валидация) на репрезентативном лейбле `toxic` - наиболее частом и достаточно сбалансированном классе, что позволило избежать чрезмерно дорогого полного перебора по всем 6 таргетам одновременно. Лучшие найденные конфигурации:

Модель	Лучшие гиперпараметры	CV ROC-AUC (лейбл <code>toxic</code>)
Logistic Regression	<code>C=3, penalty=l2, class_weight=None</code>	0.9654
Random Forest	<code>n_estimators=250, max_depth=None, min_samples_leaf=1</code>	0.9624
LightGBM	<code>num_leaves=31, n_estimators=100, learning_rate=0.05</code>	0.9354

Показательно, что уже на этапе подбора гиперпараметров по одному лейблу линейная модель не уступает более сложным алгоритмам - эта тенденция подтвердилась и на полной мульти-лейбл оценке ниже.

3.2 Кросс-валидация (5 фолдов, все 6 лейблов)

После подбора гиперпараметров каждая модель дополнительно проверена 5-фолдовой кросс-валидацией на полном наборе из 6 лейблов (усреднённый макро ROC-AUC по фолдам):

Модель	Средний макро ROC-AUC	Std по фолдам
Logistic Regression	0.9731	± 0.0026
Random Forest	0.9569	± 0.0057
Gradient Boosting (LightGBM)	0.9661	± 0.0029

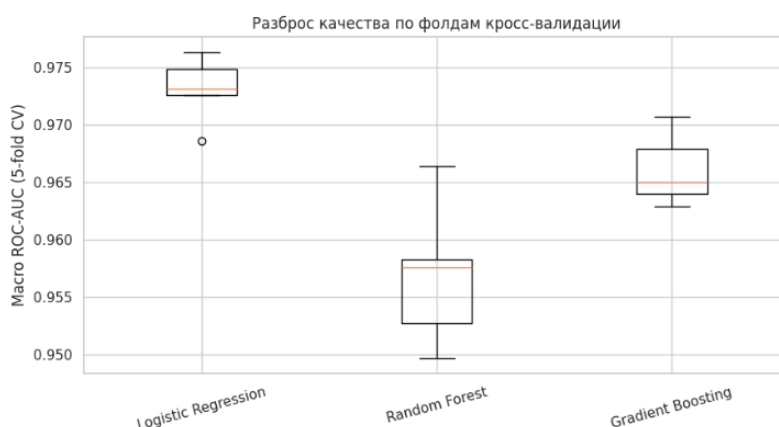


Рис. 1. Разброс качества моделей по фолдам кросс-валидации.

Logistic Regression не только показывает наибольший средний ROC-AUC, но и наименьший разброс между фолдами - модель стабильнее генерализуется на разных подвыборках, чем деревья и бустинг.

3.3 Итоговое обучение и оценка на отложенной валидации

[46]:

	model	macro_roc_auc	macro_f1	micro_f1	subset_accuracy	train_time_sec
0	Logistic Regression	0.9747	0.5348	0.6729	0.9150	18.4
3	Logistic Regression	0.9747	0.5348	0.6729	0.9150	21.0
6	Logistic Regression (optimal thresholds)	0.9747	0.6185	0.7234	0.9105	18.4
2	Gradient Boosting	0.9649	0.5424	0.6883	0.9157	794.4
5	Gradient Boosting	0.9649	0.5424	0.6883	0.9157	746.5
1	Random Forest	0.9598	0.3595	0.6288	0.9113	455.4
4	Random Forest	0.9598	0.3595	0.6288	0.9113	462.5
7	MLP (Keras)	0.9398	0.1976	0.4196	0.9017	120.4

Рис. 2. Метрики классических моделей на валидационной выборке (порог 0.5).

Наблюдения:

- Logistic Regression - лучший результат по macro ROC-AUC (0.9747) и при этом наименьшее время обучения (18-21 сек)
- Random Forest уступила по всем метрикам и обучалась на порядок дольше (455-462 сек)
- Gradient Boosting показала второй результат по ROC-AUC (0.9649) и лучший F1 при пороге 0.5 среди отдельных моделей (0.5424), но обучение заняло 746-794 сек — почти в 35 раз дольше Logistic Regression

4. Порог классификации: Precision-Recall tradeoff

Было проведено исследование порога классификации для каждого лейбла отдельно. При сильном дисбалансе классов фиксированный порог 0.5 не всегда оптимален: для редких классов (threat, severe_toxic) модель может почти никогда не пересекать 0.5, даже когда ранжирование (ROC-AUC) хорошее.

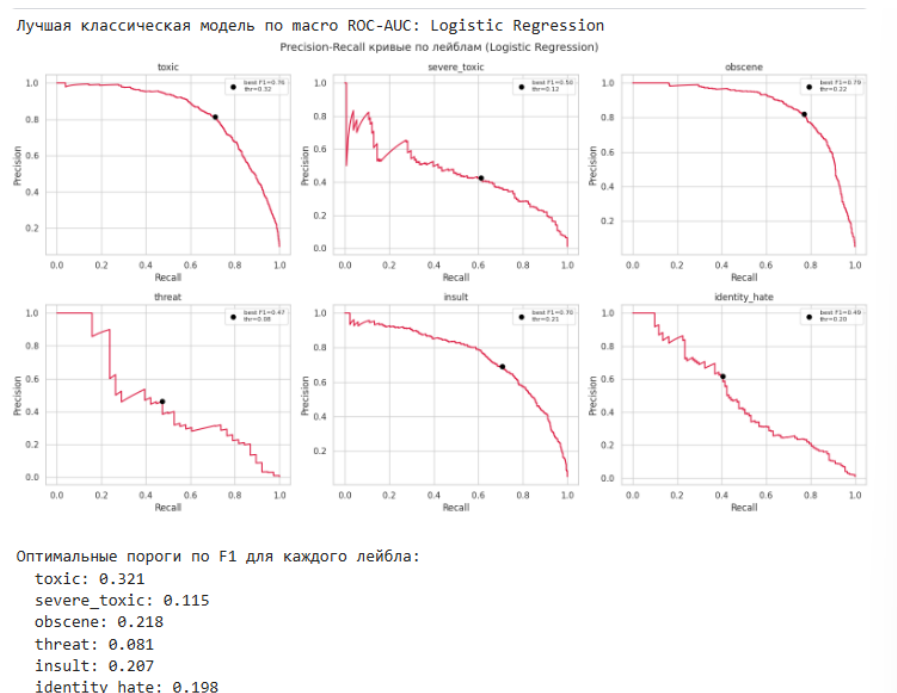


Рис. 3. Precision-Recall кривые по каждому из 6 лейблов (Logistic Regression) с отмеченным оптимальным порогом по F1

Найденные оптимальные пороги существенно ниже 0.5 для большинства лейблов: toxic - 0.321, severe_toxic - 0.115, obscene - 0.218, threat - 0.081, insult - 0.207, identity_hate - 0.198. Особенно показателен threat: оптимальный порог 0.081 - при таком сильном дисбалансе (класс встречается менее чем в 1% случаев) модель в

среднем присваивает низкие вероятности даже верно определённым позитивным примерам, и порог 0.5 отсекал бы почти все верные срабатывания.

Эффект от индивидуальной калибровки порогов ощутим на итоговой метрике:

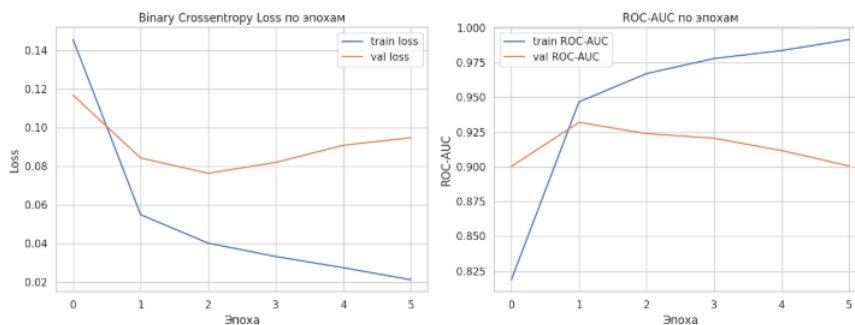
Вариант	Macro F1
Порог 0.5 (единый)	0.5348
Оптимальный порог по F1 (индивидуально по лейблу)	0.6185

5. Нейронная сеть MLP

5.1 Архитектура и обоснование

- Вход: TF-IDF (до 20 000 признаков, униграммы+биграммы) + 5 мета-признаков - плотное представление
- Скрытые слои: 256 → 128 → 64 нейронов с постепенным сжатием размерности - типичная схема для MLP на разреженных TF-IDF-входах
- Активация ReLU на скрытых слоях, Sigmoid на выходном слое (6 независимых вероятностей, а не softmax, т.к. лейблы не взаимоисключающие)
- Регуляризация: Dropout (0.3-0.4) после каждого скрытого слоя + BatchNormalization; EarlyStopping по val_auc с возвратом лучших весов
- Loss: binary_crossentropy - стандарт для мульти-лейбл задач с независимыми бинарными выходами

5.2 Кривые обучения



Если **val_loss** начинает расти при продолжающем падать **train_loss** — сеть переобучается; **EarlyStopping** в этом ноутбуке останавливает обучение по **val_auc** и возвращает лучшие веса, так что графики выше показывают, останавливаемся ли мы вовремя.

Рис. 4. Loss и ROC-AUC на train/val по эпохам обучения MLP

Графики показывают переобучение: val ROC-AUC достигает пика (~0.93) уже после 1-й эпохи и затем плавно снижается, тогда как train ROC-AUC продолжает расти почти до 1.0, а val_loss после 2-й эпохи начинает расти при продолжающем падать train_loss. EarlyStopping корректно отслеживает этот момент и останавливает обучение, возвращая лучшие веса, но даже “лучшая” точка обучения оказалась хуже классических моделей по итоговому качеству.

Итоговые метрики MLP на валидации: macro ROC-AUC = 0.9398, macro F1 = 0.1976 (при пороге 0.5), время обучения - 120 сек.

Вероятная причина: разреженный высокоразмерный TF-IDF-вход (20 000+ признаков) в сочетании с относительно небольшой обучающей выборкой (использовалась подвыборка ~60 000 строк из ~159 000 для ускорения на Kaggle) - недостаточно данных, чтобы полносвязная сеть построила устойчивое скрытое представление без переобучения, даже с Dropout и BatchNorm. Для улучшения результата стоило бы использовать плотные предобученные эмбединги (например, word2vec/GloVe/BERT) вместо sparse TF-IDF, увеличить объём обучающих данных или усилить регуляризацию (меньшая сеть, выше dropout, weight decay).

6. Сравнение моделей и выводы

6.1 Итоговая таблица метрик

2]:

	model	macro_roc_auc	macro_f1	micro_f1	subset_accuracy	train_time_sec
0	Logistic Regression	0.9747	0.5348	0.6729	0.9150	18.4
1	Logistic Regression	0.9747	0.5348	0.6729	0.9150	21.0
2	Logistic Regression (optimal thresholds)	0.9747	0.6185	0.7234	0.9105	18.4
8	Stacking (LR meta)	0.9725	0.5155	0.6745	0.9149	NaN
3	Gradient Boosting	0.9649	0.5424	0.6883	0.9157	794.4
4	Gradient Boosting	0.9649	0.5424	0.6883	0.9157	746.5
5	Random Forest	0.9598	0.3595	0.6288	0.9113	455.4
6	Random Forest	0.9598	0.3595	0.6288	0.9113	462.5
7	MLP (Keras)	0.9398	0.1976	0.4196	0.9017	120.4

Рис. 5. Полная сравнительная таблица всех моделей, включая стекинг

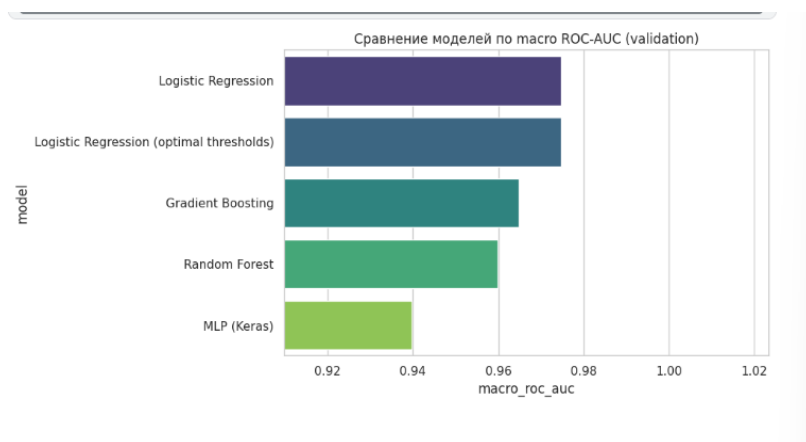


Рис. 6. Сравнение моделей по macro ROC-AUC на валидации

6.2 ROC-AUC по каждому лейблу

48]:

	Logistic Regression	Random Forest	Gradient Boosting	MLP (Keras)
toxic	0.9621	0.9606	0.9494	0.9421
severe_toxic	0.9842	0.9797	0.9787	0.9660
obscene	0.9777	0.9805	0.9782	0.9474
threat	0.9823	0.9254	0.9590	0.9336
insult	0.9728	0.9669	0.9673	0.9428
identity_hate	0.9692	0.9455	0.9570	0.9071

Рис. 7. ROC-AUC по каждому лейблу для каждой модели

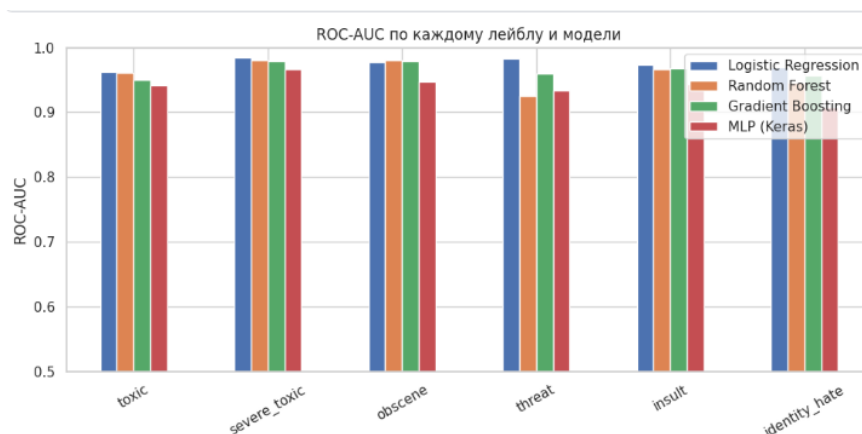


Рис. 8. То же в виде столбчатой диаграммы

Logistic Regression лидирует на 5 из 6 лейблов; единственное исключение - obscene, где Random Forest незначительно опережает линейную модель (0.9805 против 0.9777, разница 0.0028) - вероятно, признак сильнее завязан на нелинейные комбинации конкретных слов, которые дерево улавливает чуть лучше. На редких классах (threat, identity_hate) разрыв между Logistic Regression и остальными моделями особенно велик: например, threat — LR 0.9823 против RF 0.9254 и MLP 0.9336.

6.3 Анализ: почему одна модель лучше другой

- Logistic Regression оказалась лучшей моделью среди всех протестированных (macro ROC-AUC = 0.9747). Признаковое пространство TF-IDF почти линейно разделимо для явных лексических сигналов токсичности - наличие конкретных слов почти линейно предсказывает toxic/obscene, поэтому линейная модель не теряет в качестве по сравнению с более сложными, при этом обучаясь на порядок быстрее (21 сек против 462 сек у Random Forest и 746 сек у LightGBM).
- Random Forest уступила по ROC-AUC (0.9598) и особенно по F1 при пороге 0.5 (0.3595 — худший результат среди классических моделей). Деревья плохо работают с тысячами разреженных TF-IDF-индикаторов: каждое дерево видит лишь малое подмножество слов на сплит, из-за чего ансамбль хуже находит закономерности, чем линейная комбинация всех весов. Дополнительная причина низкого F1 — грубая калибровка вероятностей RF, из-за которой при фиксированном пороге 0.5 модель недоклассифицирует позитивный класс.
- Gradient Boosting (LightGBM) показала второй результат по ROC-AUC (0.9649) и лучший F1 при пороге 0.5 среди отдельных моделей (0.5424) — вероятности откалиброваны чуть лучше линейной модели. Но обучение заняло 746-794 сек — почти в 35 раз дольше Logistic Regression при худшем ROC-AUC, и потребовало заметного урезания сетки гиперпараметров, иначе полный перебор не укладывался в разумное время. Соотношение «качество/время» явно хуже, чем у линейной модели.
- MLP оказалась худшей моделью среди всех (macro ROC-AUC = 0.9398, macro F1 = 0.1976). Переобучение не удалось полностью устранить регуляризацией на разреженном высокоразмерном входе при ограниченном объёме данных.
- Редкие классы (threat, severe_toxic, identity_hate) — самые сложные для всех моделей: ROC-AUC на них ниже и более шумный между фолдами кросс-валидации из-за малого числа позитивных примеров.

6.4 Стекинг (бонус)

Стекинг (мета-модель Logistic Regression поверх вероятностей всех 4 базовых моделей) дал macro ROC-AUC = 0.9725 и macro F1 = 0.5155 — оба показателя ниже, чем у одной Logistic Regression (0.9747 / 0.5348, и тем более ниже 0.6185 при оптимальных порогах). Причина: в стекинг попала существенно более слабая MLP (ROC-AUC 0.9398), которая разбавила сигнал сильной линейной модели, а не дополнила его. Более удачным экспериментом было бы взвешенное усреднение по CV-качеству или стекинг только Logistic Regression + LightGBM, без MLP.

6.5 Feature importance

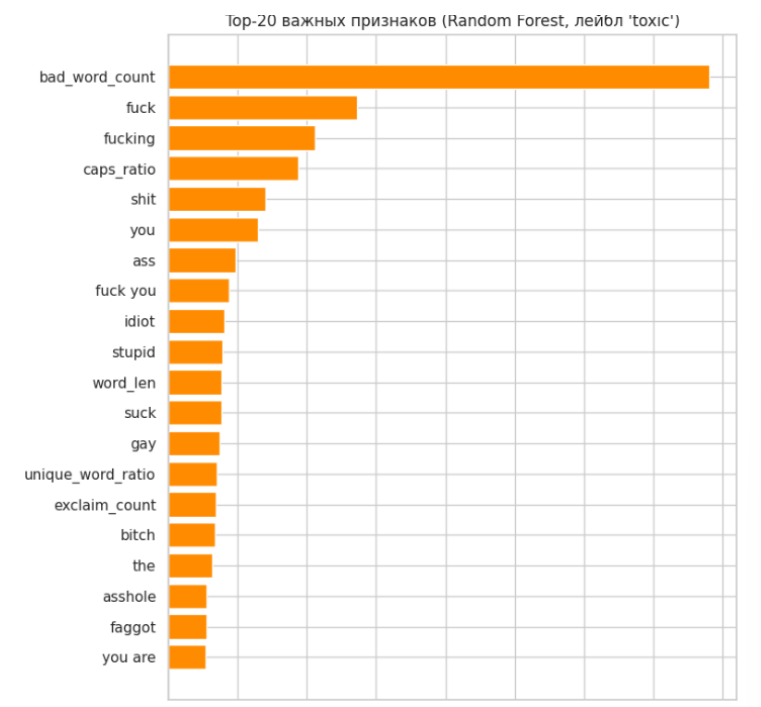


Рис. 9. Top-20 важных признаков Random Forest для лейбла toxic

Наиболее значимым признаком для Random Forest оказался инженерный мета-признак `bad_word_count` - прямой подсчёт бранных слов дал модели сильнее сигнал, чем любой отдельный TF-IDF-токен. Далее следуют конкретные бранные термины (`fuck`, `fucking`, `shit`) и `caps_ratio` - ещё один инженерный признак, что подтверждает пользу `feature engineering` из раздела 2.5, независимо от того, что сама Random Forest оказалась не лучшей моделью по итоговому качеству.

7. Содержательные выводы

1. Задача сильно несбалансирована по классам — точность (accuracy/subset_accuracy) вводит в заблуждение (у всех моделей она искусственно высокая, 0.90-0.92, даже у самой слабой MLP); корректные метрики для сравнения — macro ROC-AUC и macro F1.
2. Простая линейная модель (Logistic Regression) оказалась лучшим выбором и по качеству (macro ROC-AUC 0.9747, лучший результат на 5 из 6 лейблов), и по скорости обучения (21 сек — на порядок быстрее остальных). Для TF-IDF-признаков усложнение модели (деревья, бустинг, нейросеть) не окупилось ни по качеству, ни по затраченному времени.
3. Подбор индивидуального порога по Precision-Recall кривой дал ощутимый прирост качества лучшей модели: macro F1 вырос с 0.5348 (порог 0.5) до 0.6185 — практически бесплатный способ улучшить метрику без переобучения модели.
4. Random Forest и Gradient Boosting не оправдали вложенных вычислительных затрат: RF потратила в 22 раза больше времени на обучение, чем LogReg, и показала худший F1 среди классических моделей (0.3595); LightGBM потратила в 35 раз больше времени и всё равно уступила LogReg по ROC-AUC.
5. MLP на TF-IDF-входе без предобученных эмбеддингов переобучается быстрее и сильнее, чем удаётся скомпенсировать Dropout/BatchNorm/EarlyStopping. Для улучшения стоило бы попробовать плотные эмбеддинги вместо sparse TF-IDF, более сильную регуляризацию/меньшую сеть либо больше обучающих данных.
6. Стекинг с равным весом всех 4 моделей не улучшил результат — он оказался хуже лучшей одиночной модели (0.9725 против 0.9747 по ROC-AUC), потому что в ансамбль попала существенно более слабая MLP. Blending оправдан только когда базовые модели сопоставимы по силе.
7. Итоговая рекомендация: для продакшена — Logistic Regression с оптимальными порогами по лейблам (лучшее качество, минимальное время обучения/инференса); более сложные модели в данной конфигурации эксперимента себя не оправдали.

8. Ограничения и возможные улучшения

- Для ускорения обучения на Kaggle использовалась подвыборка ~60 000 из ~159 000 строк train — на полном датасете абсолютные значения метрик и, возможно, соотношение моделей (особенно MLP) могут измениться.
- SHAP-анализ на суррогатной Random Forest дал неинформативный результат из-за технической ошибки в обработке формата shap_values новых версий библиотеки shap (interaction values вместо обычных); после исправления código анализ следует перезапустить.
- MLP не тестировался с предобученными эмбеддингами (word2vec/GloVe/трансформеры) — это наиболее вероятное направление для улучшения нейросетевого подхода.
- Гиперпараметры моделей подбирались по одному репрезентативному лейблу (toxic), а не по всем 6 одновременно, из соображений вычислительных затрат — для редких классов оптимальные гиперпараметры могут отличаться.